

ArrivAI – Full Project Brief for Cursor

What is ArrivAI?

ArrivAI is an AI-powered migrant economic integration platform for South Korea. It helps foreigners (students, workers, migrants) find the best city in South Korea to live and work based on their skills, language level, and background – before they even arrive or right after landing.

The platform gives every user a personalized ****Integration Readiness Score (IRS)**** – a 0-100 score that tells them how economically ready they are for a specific Korean city, which jobs match their profile, and a 30-day prediction of job demand in their field.

The One-Sentence Pitch

> "ArrivAI uses real-time Korean job market data and AI to give migrants a personalized roadmap to economic independence – city by city, skill by skill."

The Problem

South Korea receives hundreds of thousands of foreign workers, international students, and migrants every year. Most arrive with skills and ambition but no clear picture of:

- Which Korean city gives them the best job opportunities
- Whether their skills are in demand right now
- What steps to take first (language, certifications, neighborhoods)
- Where demand is growing vs shrinking in the next 30 days

No tool exists that combines real job data, cost of living, and AI guidance into a single personalized platform for migrants in Korea.

Core Features

1. User Profile Input

The user enters:

- Their skills / field of work (e.g. software engineering, nursing, teaching English, manufacturing)
- Current Korean language level (None / Basic / Intermediate / Fluent)
- Country of origin
- Target city in Korea (or "suggest best city for me")

2. Integration Readiness Score (IRS)

A dynamic score from 0-100 calculated per city based on:

- Job demand in their field in that city (scraped from Saramin/JobKorea)
- Cost of living index for that city
- Language barrier weight (lower Korean = lower score)

- Community presence (how many foreigners from similar backgrounds)
- 30-day job trend (rising demand = higher score)

Display the IRS prominently as a circular gauge / animated score card.

3. South Korea City Heatmap

An interactive map of South Korea using **MapLibre GL JS** with **Maptiler free tiles**.

- Each major city (Seoul, Busan, Incheon, Daegu, Daejeon, Gwangju, Suwon, Ulsan, Jeju) is shown as a colored circle/dot
- Color = IRS score for the user's profile in that city (green = high opportunity, red = low)
- Click a city → opens a side panel with full city breakdown
- This is the visual "wow moment" – make it bold and dramatic

4. City Detail Panel

When a city is clicked:

- IRS Score for that city
- Top 5 job listings scraped from Saramin for their skill
- Cost of living index (scrape from Numbeo or static data)
- Community presence (static data is fine for hackathon)
- 30-day job demand trend (arrow up/down with % change)

5. Claude AI Assistant

A chat panel where the user can ask questions like:

- "What should I do first when I arrive in Busan as a software engineer?"
- "Which city is better for me – Seoul or Daejeon?"
- "What Korean certifications would help me get hired faster?"

Claude responds with personalized advice based on the user's profile and the real data shown on the platform.

6. 30-Day Prediction Layer

For each city, show a simple forecast:

- "Software engineer demand in Seoul is trending UP 12% over the next 30 days"
- Based on job listing volume trends from Saramin scraping
- Display as a trend bar or arrow on the city panel

Tech Stack

Frontend

- **React + Vite**
- **Tailwind CSS** for styling
- **MapLibre GL JS** + **react-map-gl** for the Korea heatmap
- **Maptiler free tiles** (get free API key at maptiler.com – no credit card needed)
- Fonts: Use something distinctive – NOT Inter or Roboto. Suggest: Sora, DM Sans, or Plus Jakarta Sans from Google Fonts
- Color theme: Dark background (#0a0f1e or similar deep navy), bright accent (electric blue or emerald green for high IRS, red/orange for low IRS)

- Animations: Smooth score counter animation, map dot pulse effect, panel slide-in

Backend

- **Node.js + Express**
- **Vercel serverless** (api/index.js entry point)
- **Vercel cron jobs** for daily data refresh

Data & AI

- **Bright Data** (BRIGHTDATA_API_TOKEN) - scrape Saramin and JobKorea for job listings by city and skill
- **Anthropic Claude API** (ANTHROPIC_API_KEY) - powers the AI assistant chat and IRS calculation logic
- **Firebase Firestore** - stores scraped job data, city scores, cached IRS calculations

Deployment

- **Vercel** - same account as MGM Assist, new project
- Backend: arrivaai-api.vercel.app
- Frontend: arrivaai.vercel.app

Folder Structure

...

```
ArrivAI/
├── server.js                # Express backend: Claude API, Bright
Data, Firestore, IRS logic
├── api/
│   └── index.js            # Vercel serverless entry → loads
server.js
├── vercel.json              # Backend deploy config + cron jobs
├── .env.example             # All env vars listed (no secrets)
├── frontend/
│   ├── index.html
│   ├── vite.config.js
│   ├── tailwind.config.js
│   ├── vercel.json         # Proxies /api/* → backend
│   ├── package.json
│   └── src/
│       ├── main.jsx
│       ├── App.jsx
│       └── lib/
│           └── api.js       # API URL builder (same pattern as MGM
Assist)
├── components/
│   ├── ProfileForm.jsx     # User input: skills, language,
origin
│   ├── IRSScoreCard.jsx    # Animated 0-100 score gauge
│   ├── KoreaMap.jsx        # MapLibre heatmap of Korea cities
│   └── CityPanel.jsx        # City detail: jobs, cost, trend,
IRS
```

```

|           |— JobListings.jsx           # Top 5 jobs from Saramin for user
skill
|           |— PredictionBar.jsx         # 30-day demand trend arrow + %
|           |— ChatAssistant.jsx         # Claude AI chat panel
|           |— Header.jsx               # ArrivAI logo + tagline
|
|— scripts/
|   |— triggerCron.js                   # Manual cron trigger for testing
|
|— package.json
...

```

Backend API Endpoints (server.js)

```

| Method | Endpoint | What it does |
|-----|-----|-----|
| POST | /api/profile | Receives user profile, calculates IRS per city,
returns city scores |
| GET | /api/jobs/:city/:skill | Returns scraped job listings for city +
skill combo |
| GET | /api/cities | Returns all city IRS scores for current user
session |
| POST | /api/chat | Claude AI assistant chat (takes message + user
profile context) |
| GET | /api/predict/:city/:skill | Returns 30-day demand trend for city
+ skill |
| POST | /api/cron/refresh | Cron job: refreshes job data from Saramin
via Bright Data |

```

Environment Variables (.env.example)

```

...

# Anthropic
ANTHROPIC_API_KEY=

# Bright Data
BRIGHTDATA_API_TOKEN=

# Firebase
FIREBASE_SERVICE_ACCOUNT_JSON=

# Maptiler (frontend – safe to expose as public token)
VITE_MAPTILER_KEY=

# Backend URL (used by frontend)
VITE_API_BASE_URL=https://arrivaai-api.vercel.app
...

```

```

## IRS Score Calculation Logic (for server.js)

```javascript
function calculateIRS(profile, cityData) {
 const { skill, koreanLevel, origin } = profile;
 const { jobDemand, costOfLiving, communityPresence, trend30d } =
cityData;

 // Weights
 const jobScore = jobDemand * 0.40; // 40% - most important
 const costScore = (100 - costOfLiving) * 0.20; // 20% - lower cost =
better
 const languageScore = getLanguageScore(koreanLevel) * 0.20; // 20%
 const communityScore = communityPresence * 0.10; // 10%
 const trendScore = trend30d * 0.10; // 10% - future demand

 return Math.round(jobScore + costScore + languageScore + communityScore
+ trendScore);
}

function getLanguageScore(level) {
 const map = { none: 20, basic: 45, intermediate: 70, fluent: 100 };
 return map[level] || 20;
}
```

```

```

## Bright Data Scraping (in server.js)

```

```

Target: **Saramin** (saramin.co.kr) - Korea's largest job board

```

```

```javascript
// Example scraping call via Bright Data Web Unlocker REST fallback
async function scrapeJobsFromSaramin(city, skill) {
 const url =
`https://www.saramin.co.kr/zf_user/search/recruit?searchword=${encodeURIComponent(skill)}&loc_cd=${getCityCode(city)}`;

 const response = await fetch('https://api.brightdata.com/request', {
 method: 'POST',
 headers: {
 'Authorization': `Bearer ${process.env.BRIGHTDATA_API_TOKEN}`,
 'Content-Type': 'application/json'
 },
 body: JSON.stringify({
 zone: 'web_unlocker1',
 url: url,
 format: 'raw'
 })
 });

 const html = await response.text();
 // Parse job listings from HTML and return array

```

```
 return parseJobListings(html);
}
```

```
// Saramin city codes
```

```
function getCityCode(city) {
 const codes = {
 seoul: '101000',
 busan: '102000',
 incheon: '108000',
 daegu: '103000',
 daejeon: '106000',
 gwangju: '104000',
 suwon: '119000',
 ulsan: '107000',
 jeju: '117000'
 };
 return codes[city.toLowerCase()] || '101000';
}
``,`
```

```

```

```
Claude AI Assistant System Prompt (in server.js)
```

```
```javascript
```

```
const ARRIVAAI_SYSTEM_PROMPT = `
```

```
You are ArrivAI's integration assistant – a friendly, practical AI guide helping migrants and foreigners find economic opportunities in South Korea.
```

```
You have access to the user's profile:
```

- Skills: {skill}
- Korean language level: {koreanLevel}
- Country of origin: {origin}
- Their Integration Readiness Score (IRS): {irs} / 100
- Their target city: {city}

```
Your job is to:
```

1. Give specific, actionable advice for their situation
2. Recommend Korean job boards, language resources, and visa info relevant to their profile
3. Compare cities honestly based on their skill and language level
4. Suggest a 30-day action plan if asked
5. Be encouraging but realistic – don't sugarcoat low IRS scores, explain why and how to improve them

```
Always respond in English. Keep answers concise and practical. Never make up job listings or statistics – refer to what the platform shows.
```

```
`;
``,`
```

```
---
```

```
## Korean Cities Data (static baseline for MVP)
```

```

```javascript
const KOREA_CITIES = [
 { id: 'seoul', name: 'Seoul', lat: 37.5665, lng: 126.9780,
 population: 9700000 },
 { id: 'busan', name: 'Busan', lat: 35.1796, lng: 129.0756,
 population: 3400000 },
 { id: 'incheon', name: 'Incheon', lat: 37.4563, lng: 126.7052,
 population: 2950000 },
 { id: 'daegu', name: 'Daegu', lat: 35.8714, lng: 128.6014,
 population: 2430000 },
 { id: 'daejeon', name: 'Daejeon', lat: 36.3504, lng: 127.3845,
 population: 1470000 },
 { id: 'gwangju', name: 'Gwangju', lat: 35.1595, lng: 126.8526,
 population: 1460000 },
 { id: 'suwon', name: 'Suwon', lat: 37.2636, lng: 127.0286,
 population: 1240000 },
 { id: 'ulsan', name: 'Ulsan', lat: 35.5384, lng: 129.3114,
 population: 1140000 },
 { id: 'jeju', name: 'Jeju', lat: 33.4996, lng: 126.5312,
 population: 670000 },
];
```

```

UI Design Direction (for Cursor / frontend)

****Aesthetic:**** Dark, data-driven, cinematic – like a mission control dashboard for your future

- Background: Deep navy `#0a0f1e`
- Primary accent: Electric blue `#00d4ff`
- Success/high IRS: Emerald `#00ff88`
- Warning/mid IRS: Amber `#ffaa00`
- Danger/low IRS: Coral `#ff4757`
- Font: ****Sora**** (headings) + ****DM Sans**** (body) from Google Fonts
- The IRS score card should animate from 0 to the final score on load
- City dots on the map should pulse gently
- City panel slides in from the right when a city is clicked
- Mobile responsive – works on phone

****Key screens:****

1. ****Landing / Profile Form**** – clean hero with tagline + input form
2. ****Dashboard**** – Korea map (left/center) + IRS score card (top right) + city panel (right)
3. ****Chat**** – floating assistant button bottom right, expands to chat panel

What Makes ArrivAI Win (the V.I.B.E. mindset)

- ****One sharp problem**** – not 4 dashboards, one focused mission
- ****Signature metric**** – IRS score is memorable and concrete

- **Prediction layer** - 30-day job demand forecast, forward-looking
- **Visual impact** - Korea heatmap is the "wow moment" for judges
- **Human story** - built by someone who navigated life as a foreigner in South Korea
- **Real data** - Bright Data scraping, not fake/static demo data
- **AI depth** - Claude assistant gives personalized roadmaps, not generic answers

Build Order for Cursor

Build in this exact order:

1. `server.js` - Express setup, /api/chat, /api/profile, /api/jobs endpoints
2. `api/index.js` - Vercel entry
3. `vercel.json` (backend) - routes + cron
4. `.env.example` - all env vars
5. `frontend/src/App.jsx` - main layout (map left, panels right)
6. `frontend/src/components/ProfileForm.jsx` - user input
7. `frontend/src/components/KoreaMap.jsx` - MapLibre map
8. `frontend/src/components/IRSScoreCard.jsx` - animated score
9. `frontend/src/components/CityPanel.jsx` - city detail
10. `frontend/src/components/ChatAssistant.jsx` - Claude chat
11. `frontend/src/components/PredictionBar.jsx` - trend indicator
12. `frontend/vercel.json` - proxy /api/* to backend
13. Firebase Firestore integration in server.js
14. Bright Data scraping in server.js
15. Final styling pass - animations, mobile responsiveness

Reference Project

This project is architecturally based on **MGM Assist** (github.com/francisnatusm/MGM-Assist) - a React + Vite frontend with Express backend on Vercel, Claude API and Bright Data on server-side only, and Firebase Firestore for data storage. Use the same patterns - especially the /api/chat endpoint structure and Bright Data REST fallback approach.

ArrivAI - Built by Francis Natus Mugisha | ML & AI Engineer | Kyungdong University, South Korea